Course Project: Health Prediction on Synthetic MIMIC-III

Zijun Yao

Assistant Professor, EECS Department

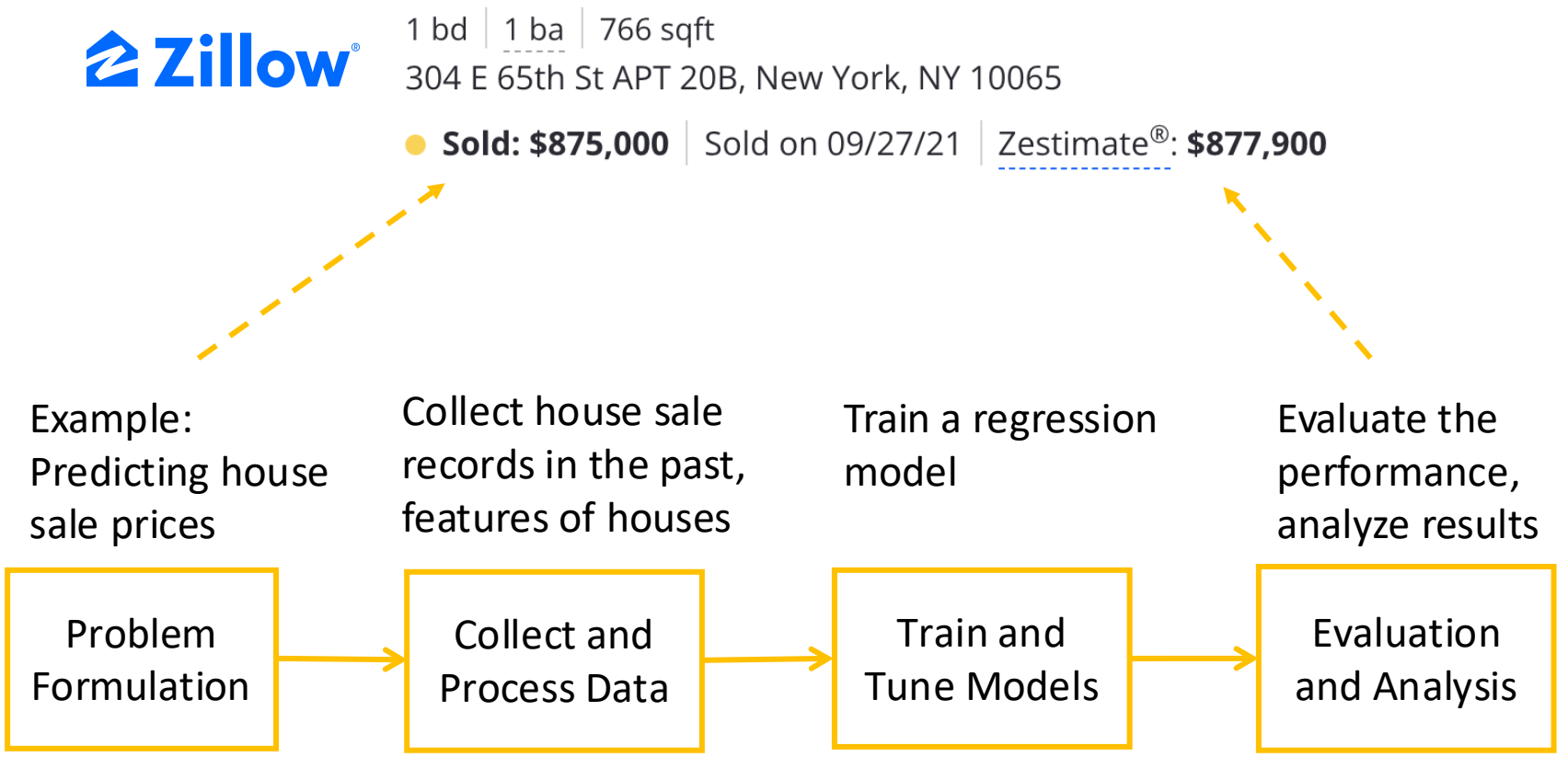
University of Kansas

Fall 2026

Outline

- Workflow and challenges of a data mining project
- Questions to ask in project development
- Course project: Health Prediction on Synthetic MIMIC-III
- Python environment setup

Project Workflow



Challenges and Time Distribution

Formulate Problem

- Convert a problem into a ML task

Data

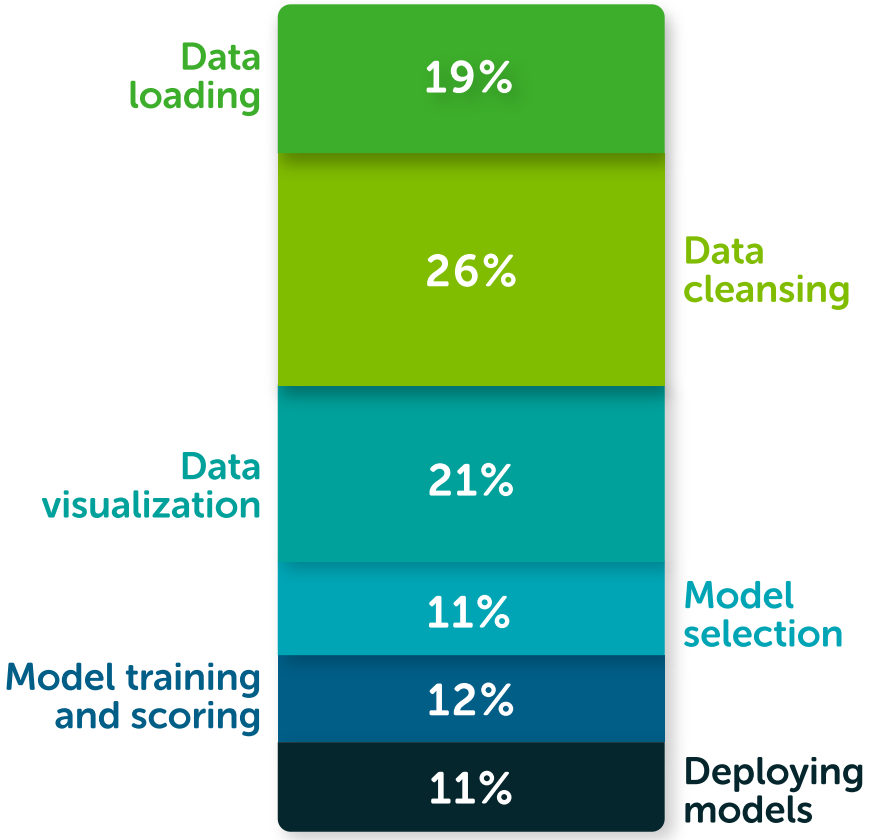
- Data collection, data exploration and preprocessing

Train Models

- Model selection, optimization, computation

Evaluation and Analysis

- Evaluation metrics, baseline, insight analysis



How data scientists spent their time?
45% of their time is spent getting data ready
21% of their time on data visualization

https://www.anaconda.com/state-of-data-science-2020

Outline

- Workflow and challenges of a data mining project
- Questions to ask in project development
- Course project: Health Prediction on Synthetic MIMIC-III
- Python environment setup

Questions to Ask in Project Development

- Formulate a data mining task?
  - Motivation for pursuing this problem
  - Background of the problem
  - What is the input and output of your framework (explicitly with features and predictions)
  - What is the models you use and why
  - What is the conclusion and insights you have got
- Understand your goal
  - What is the challenge of your study problem?
  - What is the existing method for your study problem
  - What is their strength and weakness
  - How is your work similar or different from them?

Questions to Ask in Project Development

- Dataset
  - Describe your dataset, how it is collected, feature list, data statistics (e.g., feature/target distribution, male/female, number of subjects)
  - What data preprocessing is needed (normalization, missing value, feature selection, sampling …)?
  - What are the inputs and output?
  - What is the data size (training/validation/testing)?
- Method
  - Your learning algorithms, the model, its mechanism, how it works in your scenario
  - What is the strength and weakness of your model?
  - What are the baselines methods you can use for model comparison

Questions to Ask in Project Development

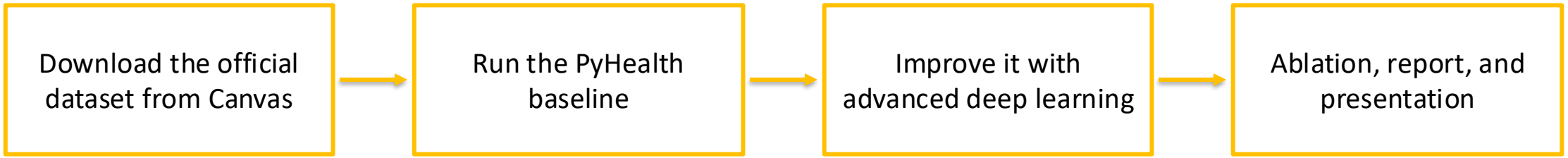
- Experiments and evaluation
  - What is the setting of your experiments and how you find it (settings for your model like learning rate, iteration, batch size)
  - Describe you training procedure (cross-validation)
  - Analyze if you have overfitting problem or other training issues and how to handle them
  - List and explain your evaluation metrics (accuracy, precision, AUC…)
  - Show results in tables and visualization
  - Show qualitative insights from of your results
  - Conclude the finding in the project and limitation of the project

Outline

- Workflow and challenges of a data mining project
- Questions to ask in project development
- Course project: Health Prediction on Synthetic MIMIC-III
- Python environment setup

Project Overview: ICU Outcome Prediction

- **Goal:** build and evaluate ML models for two outcomes per admission (in-hospital mortality, prolonged stay) on PyHealth's synthetic MIMIC-III
- **What you do:** implement a provided baseline (RNN), then improve it with an advanced deep learning method
- **Same rules for everyone:** identical data files (checksums), cohort function, 80/10/10 split (seed 42), metrics, and seeds 42/43/44
- **Team project:** groups of 2 to 4 students
- **Project support:** Chen Chen (chenchen@ku.edu)



Electronic Health Records (EHR)

- A digital record of a patient's medical history
  - Diagnoses and treatment plans
  - Medications and allergies
  - Procedures and surgical history
  - Lab results and vital signs
  - Clinical notes and imaging reports
- Longitudinal: a sequence of visits, each with timestamped events
- Prediction task: use an admission's codes to predict an outcome (e.g., in-hospital death)

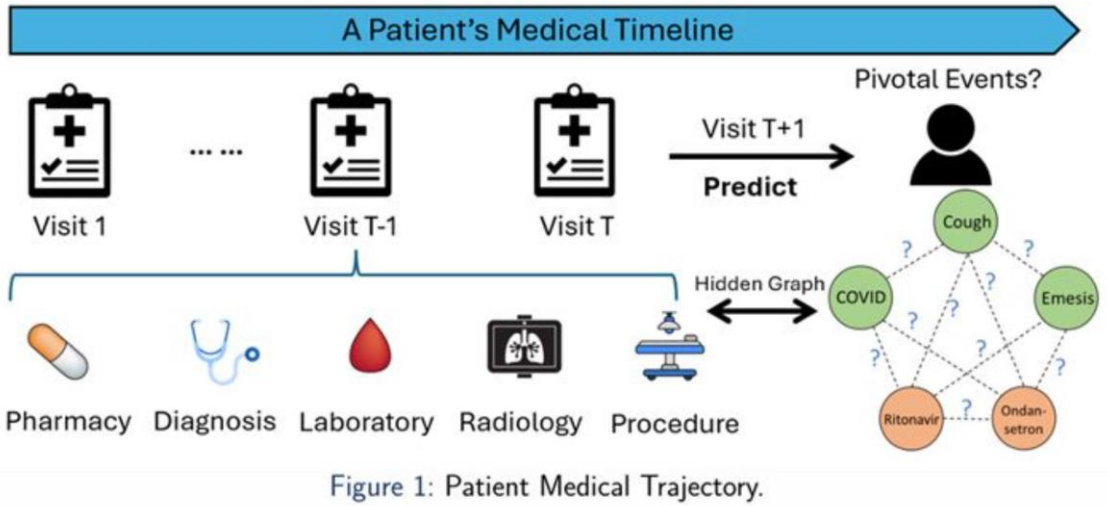
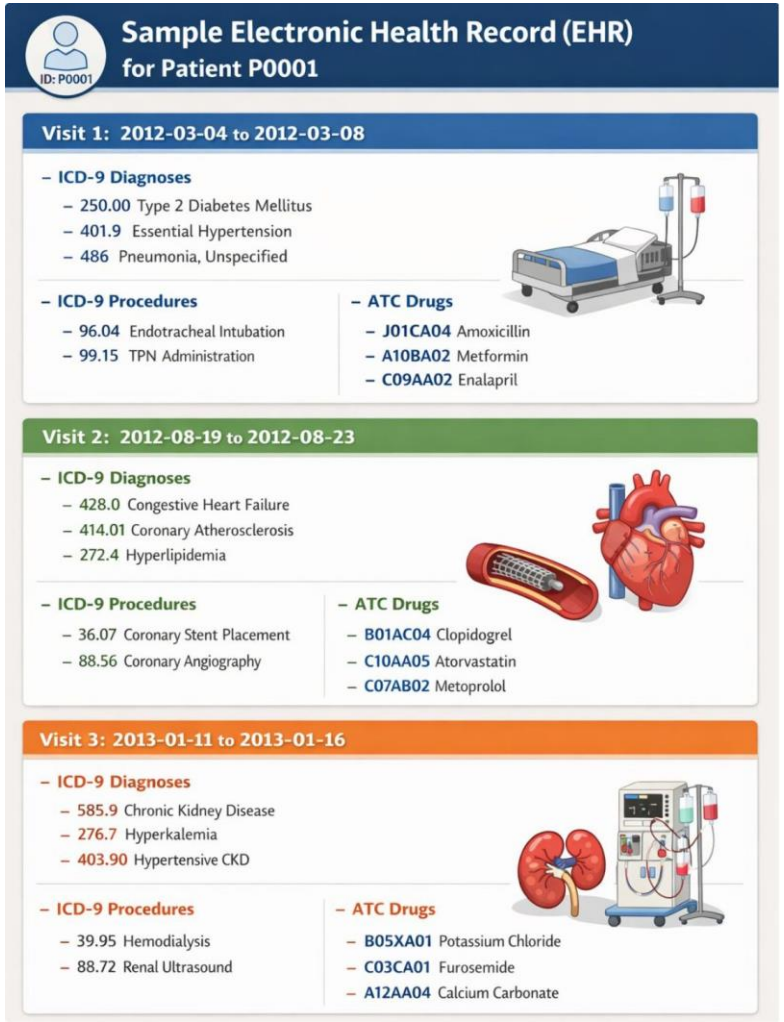


Figure 1: Patient Medical Trajectory.

What an EHR Looks Like



- Patient P0001: three hospital visits in 2012–2013
- Each visit records a set of medical codes
  - ICD-9 diagnoses (conditions), e.g., 250.00 Type 2 diabetes
  - ICD-9 procedures (performed), e.g., 96.04 endotracheal intubation
  - ATC drugs (prescribed), e.g., A10BA02 metformin
- Codes are the vocabulary a model learns from, like words in a sentence
- In MIMIC-III these live in the DIAGNOSES_ICD, PROCEDURES_ICD, and PRESCRIPTIONS tables
- Codes are sparse and high-dimensional: tens of thousands of codes, most patients have a handful

Medical Ontologies and Coding Systems

- Standardized vocabularies for clinical concepts
  - Make records interoperable across hospitals and systems
- **ICD** (International Classification of Diseases)
  - Diagnoses and procedures; MIMIC-III uses ICD-9
  - Hierarchical: 250 (diabetes) → 250.0 → 250.00
- **ATC** (Anatomical Therapeutic Chemical)
  - Drugs in 5 levels: A → A10 → A10B → A10BA → A10BA02
- Others: SNOMED CT, CPT (procedures), LOINC (lab tests)
- Why it matters: hierarchies can be used to share information among rare codes (e.g., GRAM, KAME)

Code	System	Meaning
428.0	ICD-9 dx	Congestive heart failure
401.9	ICD-9 dx	Essential hypertension
36.07	ICD-9 proc	Coronary stent placement
C10AA05	ATC	Atorvastatin
B01AC04	ATC	Clopidogrel

Dataset: Synthetic MIMIC-III (PyHealth)

- Public demo data released by the PyHealth team in the exact MIMIC-III table schema
- Six tables: PATIENTS, ADMISSIONS, ICUSTAYS, DIAGNOSES_ICD, PROCEDURES_ICD, PRESCRIPTIONS
- No real patients, no PHI, no credentialing. Absolute metric values are not clinically meaningful
- What is graded: a correct, reproducible pipeline, your method, and the evidence (baseline Δ, ablations)
- Real MIMIC-III is an optional extension only for fully credentialed teams

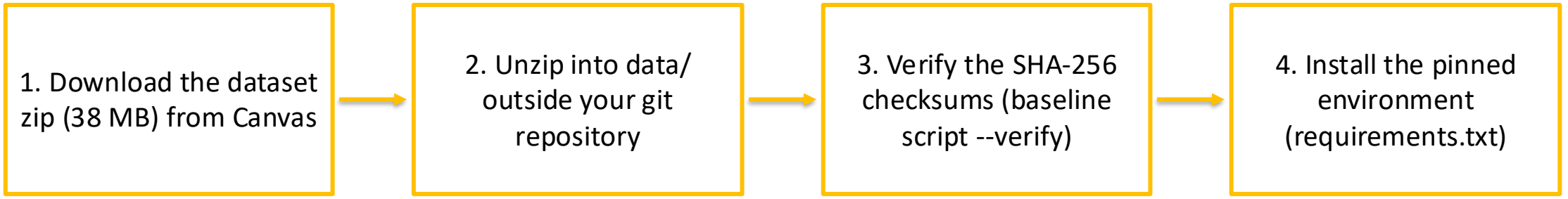
Quantity	Value
Patients	49,993
Hospital admissions	52,769
Task cohort (all three code types)	45,006
Task 1 positives: in-hospital death	1,858 (4.13%)
Task 2 positives: stay > 7 days	9,423 (20.94%)
Train / val / test (by patient, seed 42)	36,011 / 4,495 / 4,500

Your pipeline must reproduce every number above

https://storage.googleapis.com/pyhealth/Synthetic_MIMIC-III

Getting the Identical Dataset

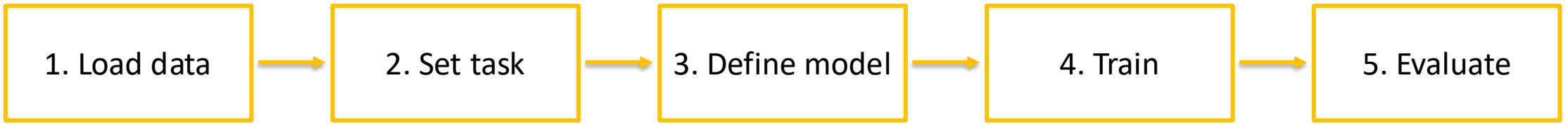
No credentialing. Everyone uses the same official copy, verified by checksum.



- **Do not modify, filter, or re-save the CSV files.** All preprocessing happens in code, so results stay comparable across teams.
- **Use the official cohort function and split.** Provided in the project PDF and the baseline script; cohort 45,006 / 1,858; split 36,011 / 4,495 / 4,500.
- **Public mirror:** the same files are at storage.googleapis.com/pyhealth/Synthetic_MIMIC-III, but the Canvas zip is the official version; the mirror may change.
- **Data statement** in your report: synthetic data, no real patient data used.

Library: PyHealth

- An open-source Python library for healthcare AI (SunLab, UIUC): no EHR parsing from scratch
  - **Dataset loader:** MIMIC3Dataset reads the six CSV tables
  - **Models:** RNN, Transformer, RETAIN, and more, with a common interface
  - **Trainer:** training loop, checkpointing, metrics (AUROC, AUPRC)
- **Pin the version: pyhealth==1.1.6.** GitHub master and the "latest" docs are 2.x with a different API
- **requirements.txt also pins** numpy<2 and setuptools<81, which fixes two install crashes
- **Reference code:** the v1.1.6 example and the provided baseline script (Canvas)
- **Reproducibility:** set_seed(seed) before the model; split_by_patient(..., seed=42)



Baseline Script (Canvas)

```
# official cohort function (PyHealth 1.1.6)
def in_hospital_mortality_fn(patient):
    samples = []
    for visit in patient:
        conditions = visit.get_code_list(table="DIAGNOSES_ICD")
        procedures = visit.get_code_list(table="PROCEDURES_ICD")
        drugs = visit.get_code_list(table="PRESCRIPTIONS")
        if len(conditions) * len(procedures) * len(drugs) == 0:
            continue   # need all three code types
        status = visit.discharge_status
        label = int(status) if status in (0, 1) else 0
        samples.append({
            "visit_id": visit.visit_id,
            "patient_id": patient.patient_id,
            "conditions": [conditions],
            "procedures": [procedures],
            "drugs": [drugs],
            "label": label})
    return samples
```

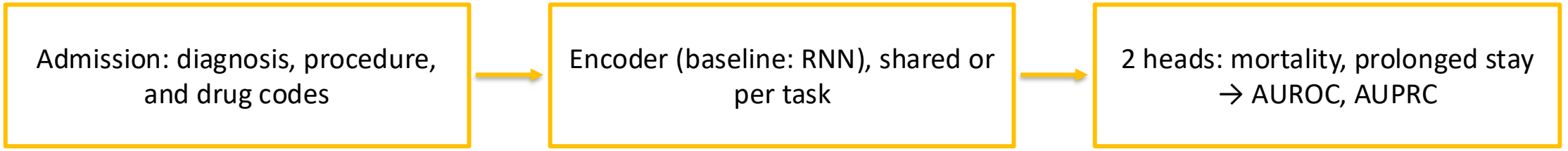
```
# one script, two tasks
python baseline_synthetic_mimic3_rnn.py \
    --task mortality --model rnn --seed 42
#   --task los ; --seed 43, 44

base = MIMIC3Dataset(root="data/synthetic_mimic3",
    tables=["DIAGNOSES_ICD", "PROCEDURES_ICD",
            "PRESCRIPTIONS"])
sd = base.set_task(TASKS[task])     # label fn
tr, va, te = split_by_patient(sd, [.8, .1, .1],
                              seed=42)
set_seed(seed); model = RNN(dataset=sd, ...)
trainer.train(..., epochs=10, monitor="roc_auc")
print(trainer.evaluate(test_dataloader))
```

- Same cohort function for both tasks; only the label function differs (see the project PDF appendix)
- Swap the RNN for your own model; keep data, split, and evaluation fixed
- 3-epoch reference AUROC: mortality 0.86, prolonged stay 0.59

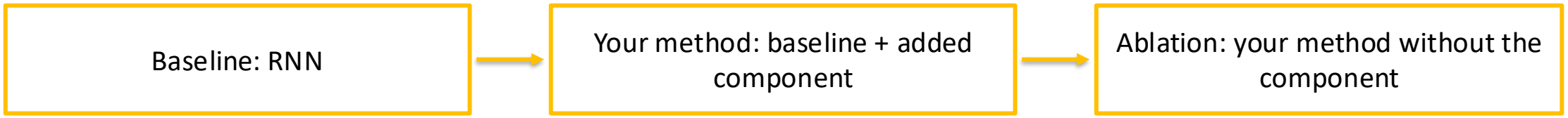
Tasks: Two Binary Outcomes per Admission

- **Shared by both tasks:** cohort of 45,006 admissions, input = the admission's diagnosis, procedure, and drug codes, split 80/10/10 by patient (seed 42), AUROC and AUPRC, seeds 42/43/44
- **Task 1, in-hospital mortality (primary):** HOSPITAL_EXPIRE_FLAG; 1,858 positives (4.1%)
- **Task 2, prolonged stay:** DISCHTIME − ADMITTIME > 7 days; 9,423 positives (20.9%)
- **Models:** one shared backbone with two heads, or two separate models
- **Expect:** mortality is learnable (baseline AUROC 0.86); stay length is weakly related to codes in the synthetic data (0.59). An honest null result with analysis earns full credit
- **Forbidden inputs:** any field that encodes an outcome or the end of stay (DEATHTIME, DISCHTIME, DISCHARGE_LOCATION, LOS, DOD)



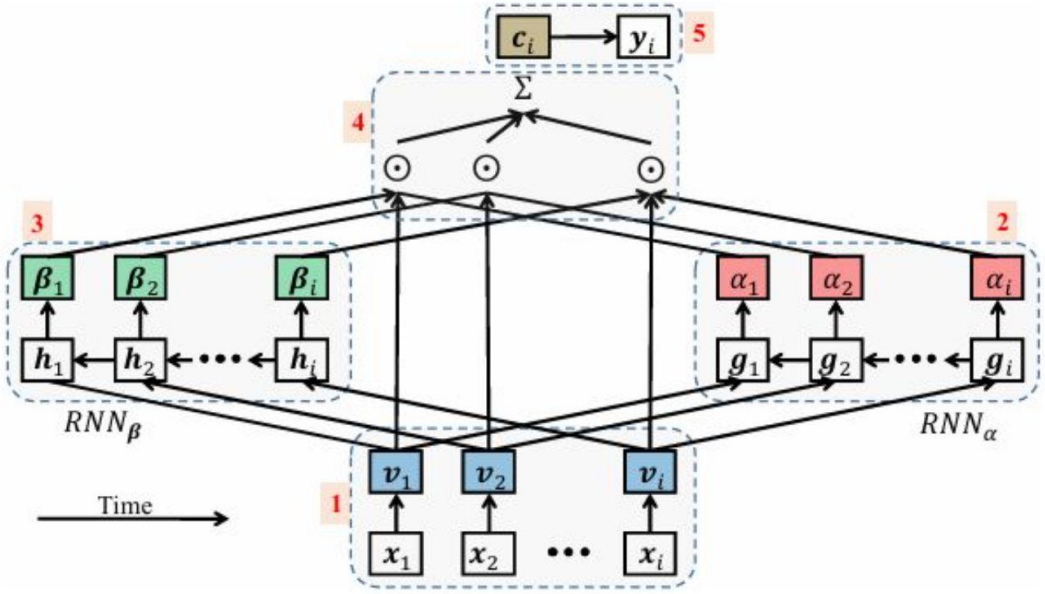
Method: Advanced Deep Learning

- **Goal:** beat the RNN baseline on both tasks with a more expressive encoder for visit or event sequences (mortality is primary)
- Pick one direction (all available or easy to build in PyHealth)
  - **Attention / Transformer encoders** over visit sequences (e.g., BEHRT)
  - **Reverse-time attention** (RETAIN): interpretable visit- and code-level attention
  - **Graph / hypergraph models** over code co-occurrence (graph transformers on EHR)
- **Minimum deliverables**
  - (1) Clear architecture description
  - (2) At least one ablation that removes the added component
  - (3) Training and inference cost discussion: accuracy gains vs. efficiency trade-offs



Example: RETAIN (Reverse Time Attention)

- Two RNNs run backward in time and produce attention weights
  - α: which visits matter (visit-level)
  - β: which codes within a visit matter (feature-level)
- Context vector: weighted sum of visit embeddings → prediction
- Interpretable: contribution score of each code at each visit
- Already implemented: pyhealth.models.RETAIN
- Ablation idea: remove β (or α) and measure the drop



Choi et al. RETAIN: An interpretable predictive model for healthcare using reverse time attention mechanism. NeurIPS 2016.

Deliverables

Code (reproducible)

- One-command training for the baseline and your method, configs included
- Seeds 42, 43, 44; logs; checkpoints; metric scripts; README
- Data check (--verify) and data/ in .gitignore

Report (6–10 pages)

- NeurIPS 2024 LaTeX template (Overleaf)
- Baseline vs. method on both tasks; ablation on mortality; mean ± std over 3 seeds
- Limitations and data statement

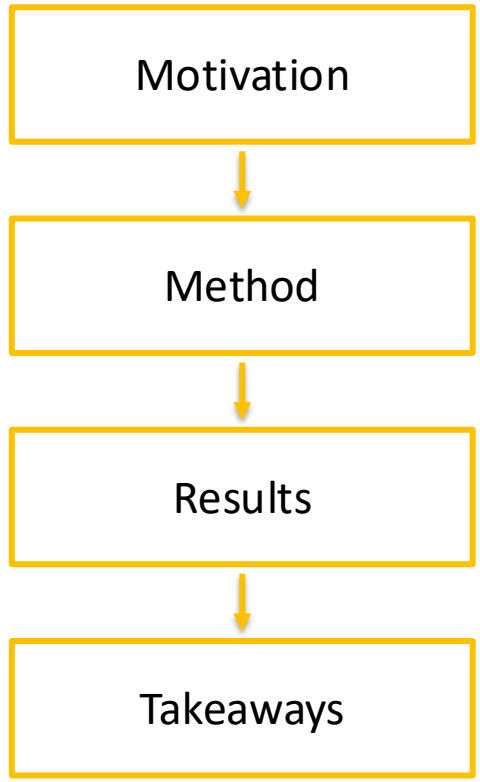
Presentation (8–10 min)

- Motivation → Method → Results → Takeaways
- Show the key tables and one insight
- Every team member participates

Report Structure and Presentation

1. Task definition and cohort construction
2. Baseline description and results
3. Your method and design choices
4. Main results table (baseline vs. method)
5. Ablations
6. Optional: error analysis (case studies)
7. Optional: efficiency (time, memory, hardware)
8. Limitations and data statement (synthetic data)

Presentation flow (8–10 min)



Template: https://www.overleaf.com/latex/templates/neurips-2024/tpsbbrdqcmsh

Grading Rubric (110 points)

Component	Points	What we look for
Correctness & Reproducibility	25	Runs on the official data and reproduces the reference cohort; no leakage (fixed split, no forbidden inputs); pinned environment, 3 seeds, logs and checkpoints (10)
Method Quality	25	Technical soundness; justified design choices
Evidence & Analysis	20	Baseline-vs-method table with Δ for both tasks, mean ± std over 3 seeds (headline Δ on mortality; an honest null result on task 2 is accepted); at least one meaningful ablation on mortality
Report Clarity	15	Clear writing, readable tables and figures, claims supported by evidence
Presentation	15	Motivation → method → results → takeaways in 8–10 minutes
Extra Credit (up to 10)	10	Robustness to missingness or sparsity (stress tests); interpretability (attention or feature attribution, error case studies, auditing)

Required Tables in Your Report

Table 1: Baseline comparison, both tasks (required)

Model	Mortality AUPRC	Mortality AUROC	Prolonged stay AUPRC	Prolonged stay AUROC
Baseline (RNN)				
Your Method				
Δ (Method − Baseline)				

Table 2: Ablation study on mortality (required)

Model Variant	AUPRC	AUROC
Your Method (Full)		
Full w/o Component 1		
Full w/o Component 2		

- Every cell is mean ± std over seeds 42, 43, 44 on the test split
- Official cohort and fixed split for every row
- Headline Δ is judged on mortality; an honest null result on prolonged stay is fine
- Define each "Component" in the method section (e.g., attention module, positional encoding, auxiliary loss); ablation on task 2 is optional

Key Reminders

Form your team now

2 to 4 students. Download the dataset from Canvas, run --verify, and run the baseline this week.

Identical data

Official zip, checksums, cohort function, and split seed 42 for everyone. No credentialing. Data statement in the report.

Reproducibility first

Pinned environment. Seeds 42, 43, 44 with mean ± std. One-command training with configs, logs, and checkpoints.

Ablation required

At least one ablation on mortality that removes your added component. Report both tasks, mean ± std over 3 seeds.

Outline

- Workflow and challenges of a data mining project
- Questions to ask in project development
- Course project: Health Prediction on Synthetic MIMIC-III
- Python environment setup

Why Python for Data Science

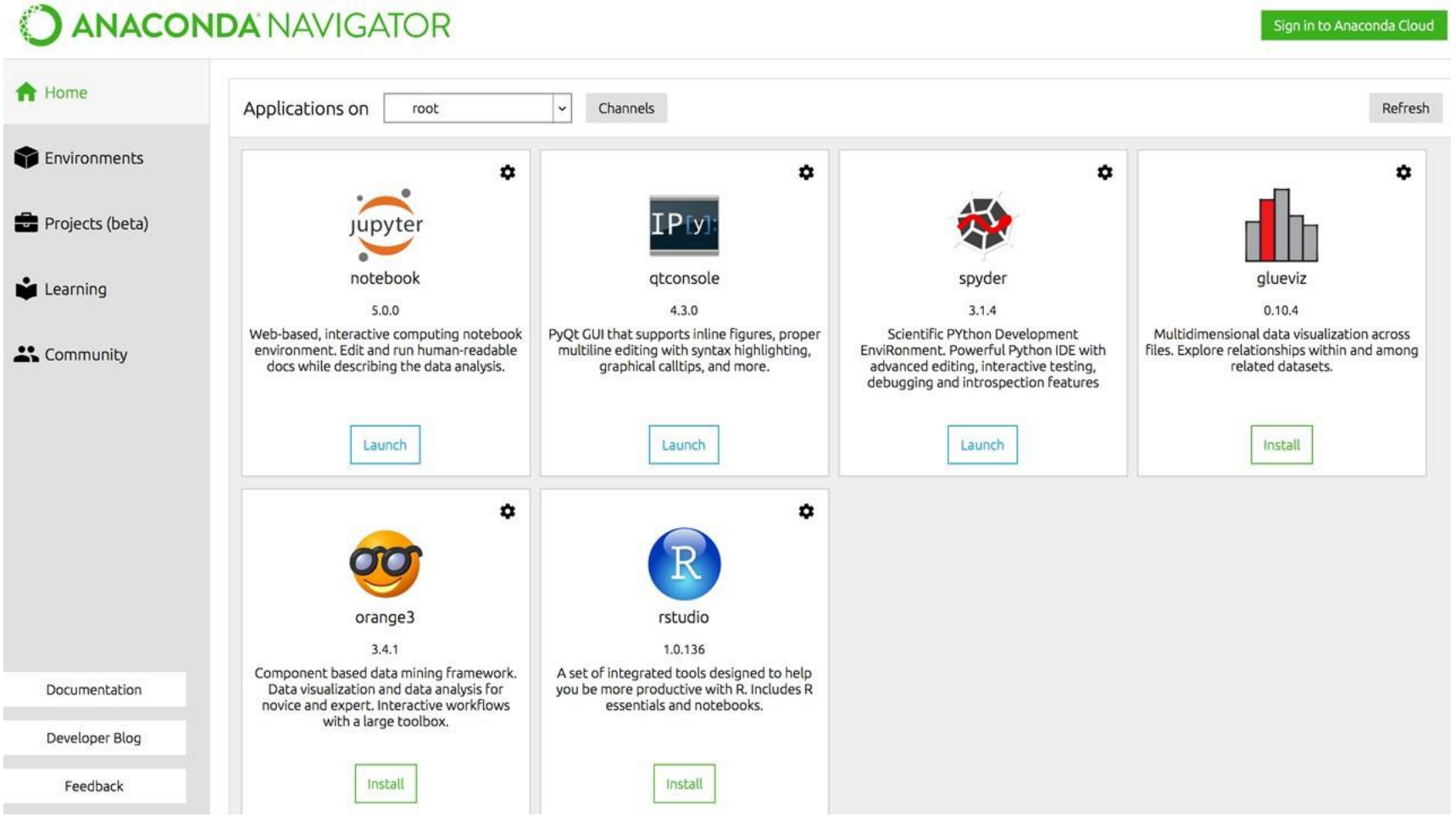
- Simplicity and Scalability
  - Open-source, high-level, interpreted, fast in writing
- Libraries
  - Numpy, Pandas, Scikit-learn, Pytorch, Keras …
- Huge Community
  - Regular releases, complete documentation, latest technology
- Jobs and Growth
  - Data scientist: Python (87%), R (31%), Java (15%)
  - Recommended most: Python (79%), R (9%), C++ (1.4%)

https://businessoverbroadway.com/2020/06/29/usage-of-programming-languages-by-data-scientists-python-grows-while-r-weakens/
https://towardsdatascience.com/top-10-reasons-why-you-need-to-learn-python-as-a-data-scientist-e3d26539ec00

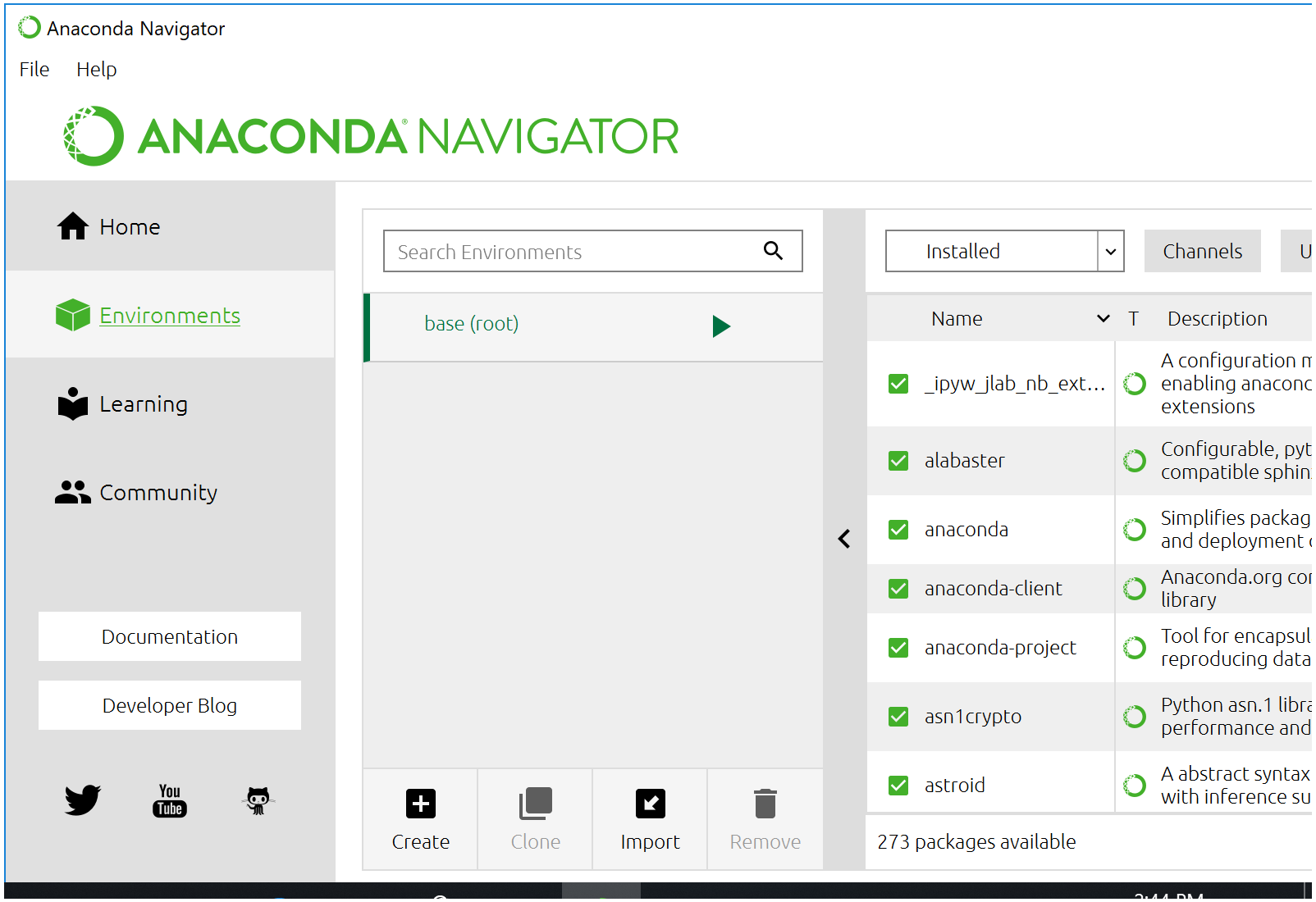
Conda: Environment Management Tool

- Managing libraries in Python can be complicated
  - Package versions for different projects are independently maintained
  - The entire environment is easy to monitor and transfer
  - Version dependency is automatically managed
- Easy to use:
  - Anaconda Navigator (GUI)
  - Terminal (Mac/Linux)
  - Anaconda prompt (Windows)
- Always use **conda** for environment management
- **Anaconda** – an integrated platform including Python, conda, and meta packages.

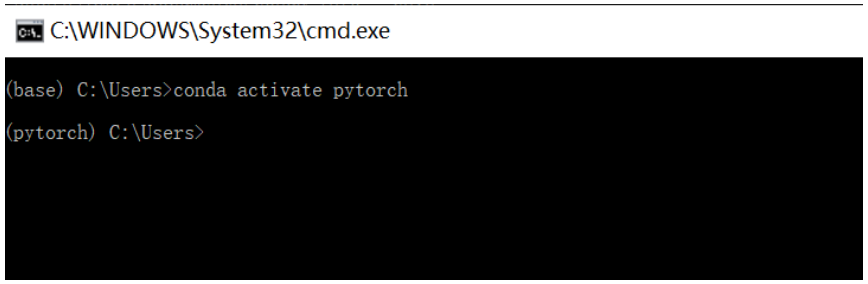
Anaconda Navigator



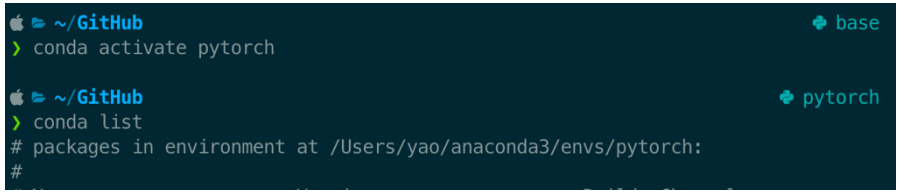
Anaconda Navigator



Conda Command Line



Windows Anaconda Prompt



Mac Terminal

Create a new environment

`conda create -n name1 python=3.9`

Create an environment (from configuration)

`conda env create -f environment.yml`

Activate an environment after creation

`conda activate name1`

List existing environments

`conda env list`

Install a package in current environment

`conda install PACKAGENAME (or pip)`

e.g., numpy, pandas, scikit-learn

https://conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html

Setting Up the Project Environment

Create and activate a dedicated environment

```
conda create -n eecs568 python=3.11
conda activate eecs568
```

Install the pinned requirements (Canvas)

```
pip install -r requirements.txt
# pyhealth==1.1.6 torch numpy<2 pandas<2 setuptools<81
```

Check the installation and the data

```
python baseline_synthetic_mimic3_rnn.py --verify
```

Save the environment for your teammates

```
conda env export > environment.yml
```

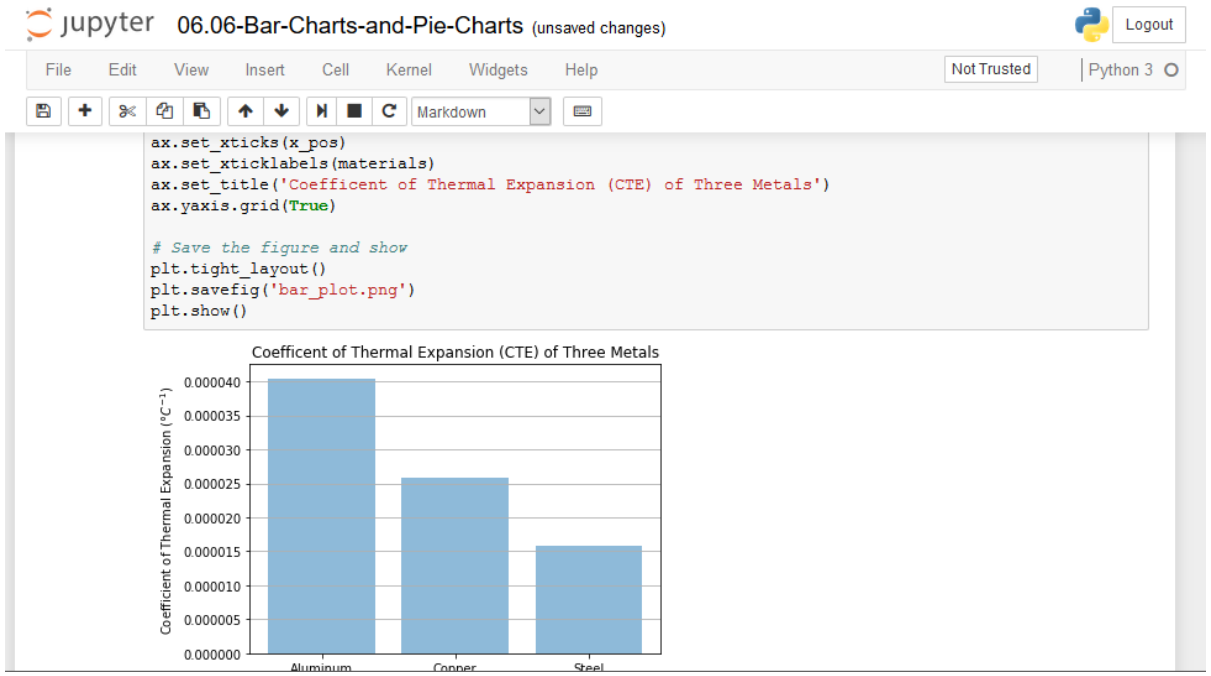
Tips

- Run the provided baseline first (about 30 s per epoch on a laptop CPU); then swap in your model
- Keep data/ outside the git repository or in .gitignore; never commit the CSV files
- A GPU helps for Transformer models, but the baseline runs on CPU
- Call set_seed(seed) before building the model; keep the split at seed 42; configs in files, not in code

https://pyhealth.readthedocs.io/en/latest/

Notebook: Interactive Coding

- Notebook
  - Interactively coding and showing results step-by-step, cell-by-cell
  - Easy to visualize data, compatible with pandas and matplotlib
  - Easy to collaborate, transfer, interpret and validate others' code
  - Easy for codes documentation using markdown.



Notebook: Friendly for Code Review

- Notebook code file can be easily reviewed in Github
- Printed as html file for review:

2. Python

https://www.w3schools.com/python/

2.1 Basics

In [2]:
```
# input and output
name = input()
print("hello, " + name)
```

```
EECS690
hello, EECS690
```

In [3]:
```
print("print with new line")
print("print without new line", end="")
print()
print("print multiple variables separated by a space:", name, 1, 3.0, True)
```

```
print with new line
print without new line
print multiple variables separated by a space: EECS690 1 3.0 True
```

Your Friends for Debugging

Python Command	Description
array.shape	Get shape of numpy array
array.dtype	Check data type of array (for precision, for weird behavior)
type(stuff)	Get type of a variable
import pdb; pdb.set_trace()	Set a breakpoint (https://docs.python.org/3/library/pdb.html)
print(f'My name is {name}')	Easy way to construct a message

Resources

- Anaconda Installation
  - Mac: https://docs.anaconda.com/anaconda/install/mac-os/
  - Windows: https://docs.anaconda.com/anaconda/install/windows/
- Live Demo: python_and_pandas.ipynb (Canvas)
- Project
  - Project description PDF and announcements: Canvas
  - Dataset: synthetic_mimic3_eecs568.zip, requirements.txt, baseline script (Canvas); public mirror: https://storage.googleapis.com/pyhealth/Synthetic_MIMIC-III
  - PyHealth 1.1.6 example: github.com/sunlabuiuc/PyHealth/blob/v1.1.6/examples/mortality_mimic3_rnn.py
  - Report template: NeurIPS 2024 on Overleaf